

SIMATIC SD 格式的 LAD 程序代码表示



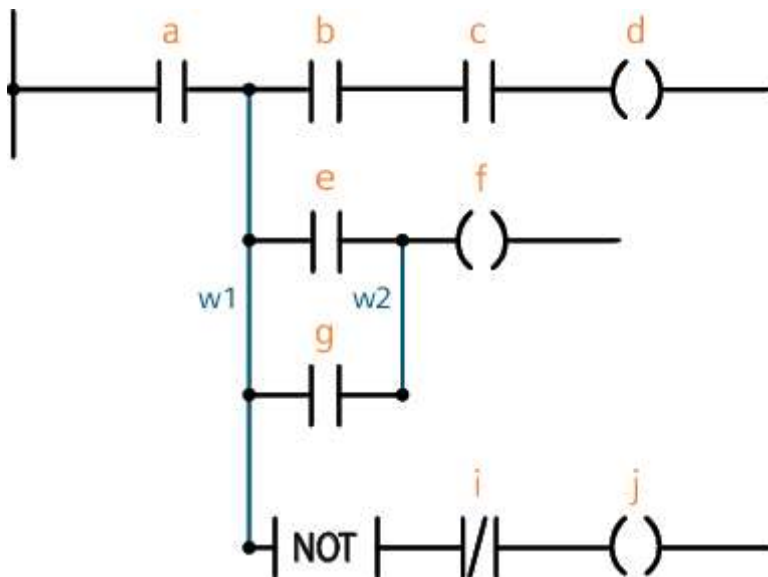
SIMATIC SD 格式的导出编译机制

SIMATIC SD 格式的 LAD 块的文本表示基于 SCL 程序代码。与使用 SCL 编程相比，编译 SIMATIC SD 格式的图形块时，两个附加关键字至关重要。这些关键字为“RUNG”和“END_RUNG”。

至少一个梯级分支从程序段的母线分支出来。通过分支，一个梯级可并联多个梯级。每个梯级都由关键字 RUNG 进行标识。如果 RUNG 从母线开始，则命名为 RUNG wire#powerrail。如果 RUNG 从分支开始，则需为该分支指定一个标识符。导出 SIMATIC SD 格式时，这些标识符由系统自动生成。其名称可以由字母、符号和数字组合构成。系统默认，分支按照发生顺序以 w1、w2、w3 等方式进行命名。名称 RUNG wire#w1 表示 RUNG 从标识符为 w1 的分支开始。要完成 RUNG，需使用 END_RUNG 语法。如果 RUNG 以关闭分支结束，则语法为 END_RUNG wire#w1。触点和线圈从左到右排列在一个梯级上。在文本表示中，触点和线圈按照从上到下的顺序显示 RUNG 列表中。

可导入的 LAD 程序的构建原理：

- 梯级数量与 RUNG 的数量相对应。
- 在 RUNG 中，触点和线圈显示为一个列表。
- RUNG 中所列的各个框，分别关联 EN 或 IN 参数。
- 所有分支均使用一个标识符（w1、w2、w3 等）进行命名。
- 各分支按照各自的发生顺序进行编号。



在本示例中，包含 4 个梯级和 2 个分支。第一个梯级从母线开始，导出为 SIMATIC SD 格式时命名为 powerrail。之后的 3 个梯级从第一个分支开始，因此这些 RUNG 的标识符为 w1。第三个梯级以第二个分支进行结尾，标识符为 w2。



RUNG wire#powerrail

```

Contact( #a )
wire#w1
Contact( #b )
Contact( #c )
Coil( #d )
END_RUNG
RUNG wire#w1
Contact( #e )
wire#w2
Coil( #f )
END_RUNG
RUNG wire#w1
Contact( #g )
END_RUNG wire#w2
RUNG wire#w1
Not()
I_Contact( #i )
Coil( #j )
END_RUNG

```

所有 LAD 块均可导出为 SIMATIC SD 格式。无法编译的差块也可导出。用户可随时生成文本文件，并保存在版本控制系统中。

常规结构

文本文件遵循程序块的结构。引用块常规属性的附注首先显示在大括号中。

```

{
  S7_Optimized := "TRUE";
  S7_PREFERREDLanguage := "LAD";
  S7_Version := "0.1"
}

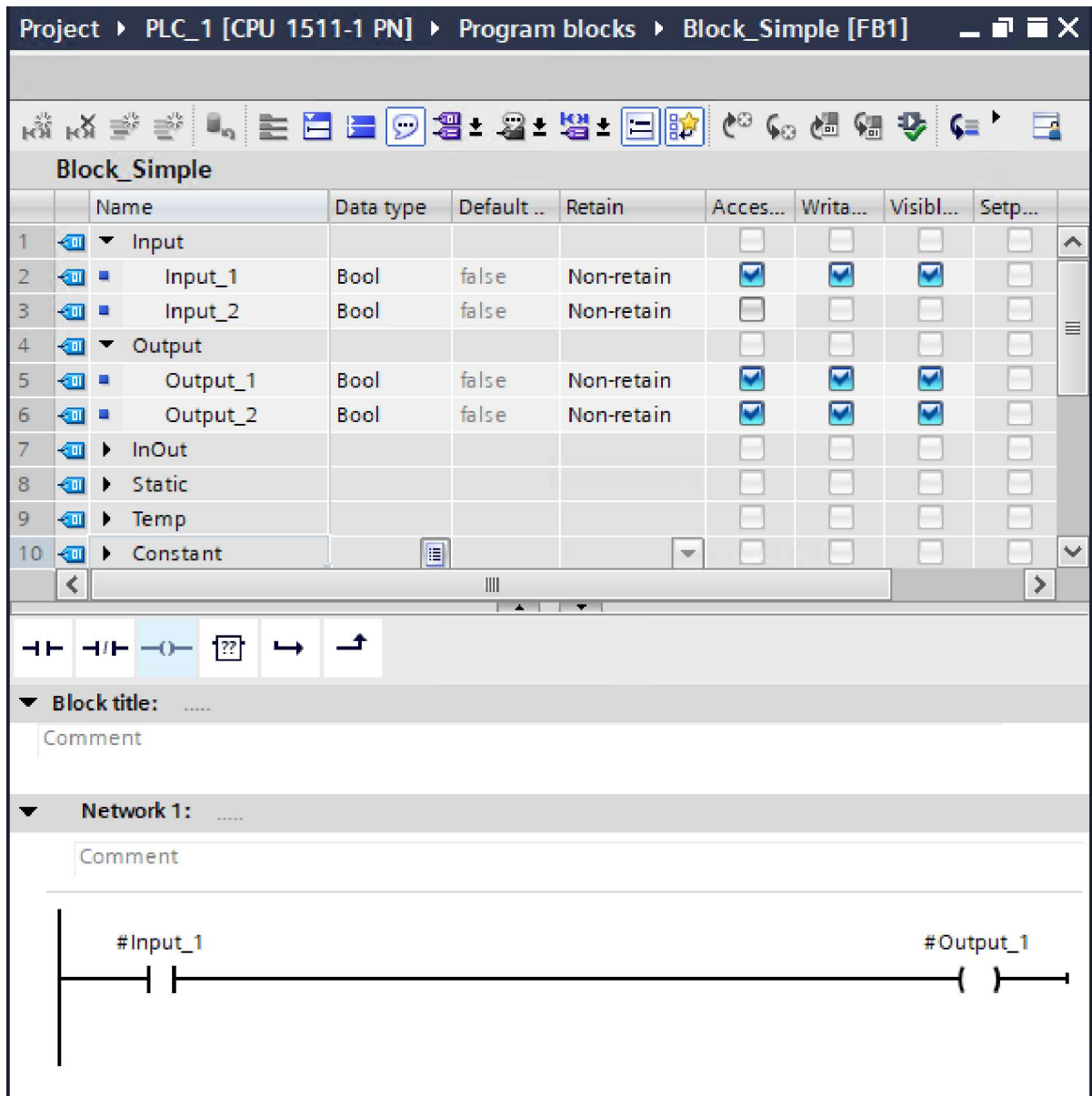
```

以下 **FUNCTION_BLOCK** 部分包含描述块内容的块名称（从块接口开始）。有关块接口的文本描述的结构和规则，请参见[此处](#)。各个程序段的程序代码如下。

该块仅通过名称来寻址，其在代码中的关键字 **FUNCTION_BLOCK** 后指定。该块的编号未显示在文本文件中。

示例

在此，将显示导出为 SIMATIC SD 格式后 TIA Portal 中的 LAD 块。



从 TIA Portal 以 SIMATIC SD 格式导出该块时，将生成以下文本表示结果：



```

{
    S7_Optimized := "TRUE";
    S7_PREFERREDLANGUAGE := "LAD";
    S7_Version := "0.1"
}
FUNCTION_BLOCK "Block_Simple"
VAR_INPUT

    Input_1 :Bool;
  
```

```
END_VAR
VAR_OUTPUT
    Output_1 :Bool;
END_VAR

{ S7_Language := "LAD" }
NETWORK
RUNG wire#powerrail
Contact( #Input_1 )
Coil( #Output_1 )
END_RUNG
END_NETWORK
END_FUNCTION_BLOCK
```